**Prepared 2026-11-13 · developer**

Development web source only. Module availability and hosted services require separate acceptance. Native and production deployment are separate.

`Source bcbf8da1f91c2101a125327f3c3be6dda77ec561`

`Application 8a3294f2724bcfe06737c0b933064fa6326fb6c9784c664810cf6b50e69cd167`

# NEVORA web developer guide

This distribution describes the accompanying development source. Its manifest binds source and guide bytes; it does not establish provider activation, hosted acceptance or production authorization. The broader developer reference supplies additional contract detail. Native apps and native push remain a separate track.

## Establish your working source

Read AGENTS.md, the relevant repository skills, package.json, tests and the code you intend to change. Fetch and reconcile `origin/development` before new integration work. Inspect status and ownership; preserve other agents' changes and use a detached worktree for isolation. Only `development`, `main` and `claude-codex-collab-nevora` are active branches under the current policy. Main and production require Jose's explicit authorization.

Run `npm run check:development-sources` after fetching before assembly. Its inventory detects drift; it does not merge or approve sources. Record exact candidate identity, changed scope, checks actually run and remaining gates in the handoff.

## Architecture and source ownership

The app uses Next.js App Router, React, TypeScript and PostgreSQL/Neon. The current company model has one company per database, a singleton workspace, locations and explicit employee eligibility. Isolated-company onboarding uses a separate control registry and guarded runtime bindings; its existence does not authorize changing an existing single-company deployment.

| Layer | Responsibility and entry points |
|---|---|
| Protected shell and pages | `src/app/app/layout.tsx`, `src/app/app/**/page.tsx`, components and nearby server actions |
| Transport | `src/app/api/**/route.ts` and domain HTTP modules validate envelope, origin, size and inputs |

| Layer | Responsibility and entry points |
| --- | --- |
| Identity and authority | `src/lib/auth/`, `src/lib/company-access/`; obtain current server session/proof and current scope |
| Business rules and commands | Scheduling, shift changes, time corrections, payroll, notifications, onboarding and Owner-advisor modules under `src/lib/` |
| Persistence | `src/lib/data/`, guarded domain stores and `src/lib/db.ts` |
| Installation/runtime | `db/migrations/order.ts`, company setup and `src/lib/tenancy/`; use recorded migration/source contracts |
| Private operations | `src/lib/operations/`, `ops/workforce/`, canonical worker entry points |

Trace a real command end to end before editing. A typical path is a form/server action or API route, parser/service, guarded transaction and bounded response or receipt. Presentation and pure validation do not establish database authority. Keep business decisions, transport, UI and data access separate. Claude owns product UI; coordinate a contract before changing that surface.

## Preserve these contracts

- **Current authority:** `OWNER`, `MANAGER`, `LEAD` and `EMPLOYEE` are access roles, not positions. Manager operational scope is an Owner-assigned current company/location grant. Payroll grants and financial-object coverage are separate. Recheck authority at protected read/write boundaries and across awaited work; navigation filtering alone is insufficient.
- **Receipts and uncertainty:** retain the original operation ID and immutable command when recovering an uncertain result. A timeout can follow a committed write. Use the workflow's own recovery protocol; not every mutation implements the same receipt contract.
- **Reviewed versions:** preserve expected revisions, source fingerprints and review tokens. Do not refresh tokens behind the user to force a stale command through. Drafts, published schedule snapshots and acknowledgements are separate.
- **SQL versions:** preserve microsecond timestamp text under the owning transaction's UTC, ISO/YMD and READ COMMITTED contract. Follow the upgrade compatibility procedure for Owner invitation reissue and fresh review. A merge performs no hosted migration.
- **Time and money:** calendar labels differ from instants and elapsed time. Use the workspace timezone and existing helpers. End-of-day `24:00` differs from the start-of-day `00:00`. Preserve exact payroll minor units, rounding and recorded-source rules.
- **Private reads:** ready-empty, unavailable, denied and expired are distinct. Private browser snapshots expire and clear on access/lifetime changes. Ignore late results from an older navigation, session or request.

- **Feature gates:** gate pages, actions and APIs as well as navigation. The communications D04 hold requires owned disposable local data; a feature flag is not a live activation bypass.
- **Rondo:** model tools read bounded evidence. Coverage mutations require the explicit Owner-reviewed command and commit-time checks. Saved goals are intent, not authority. Provider configuration does not prove provider authentication.

## Onboarding and worker boundaries

Production admission defaults closed. The approved-recipient mode is a bounded inbox fence, not public signup. Mixed open admission plus a production inbox fence fails closed. Registration rejection precedes persistence. Provider creation requires explicit capacity policy, including the bounded total-project cap; pending/unknown attempts retain capacity until safely reconciled.

Keep the web, provisioner, enrolled worker and independent monitor principals separate. Do not expose privileged bootstrap or management credentials to the browser/web process. Fresh-company installation, existing-company upgrade and restore are different operations; never rerun the fresh seed chain on an existing company. See self-service operations and worker enrollment.

## Verify safely

Inspect scripts before execution and use the version/locked dependencies in the current CI configuration. The declared `build` and `start` run Next.js directly; setup, workers and database harnesses have separate side effects. Read the engineering-toolkit README before using shared utilities.

For an ordinary code change, run the relevant focused regression first, make the smallest fix, and rerun it. The normal static/unit checks are:

```
npm test -- --maxWorkers=1
npm run typecheck
npm run lint
```

Use the specific owned PostgreSQL/browser runner for integration evidence. Read its ownership and cleanup controls first; use fictional, invocation-owned data. Never point tests at customer databases or use real SMTP/push as a fixture. A worker invocation can create jobs, documents or deliveries; it is not a harmless health probe. Do not run migrations by filename order.

Follow CI for source-bound builds and SQL/browser/package gates. Phase E requires six leaves and all 70 combinations: separate Home engines run seven widths each; other suites run both. Existing limits apply; missing, duplicate or mismatched evidence fails aggregation. Preserve failures, skips and independent findings. Record source SHA, environment, command, result and artifact identity; distinguish local, CI, development, hosted and production acceptance.

For guides, validate links/evidence, rebuild and verify PDFs with checked-in tooling. Never relabel older PDFs or hosted receipts. Keep secrets and private test content out of Git, logs and support records. See the operations guide for deployment custody and incident boundaries.